



# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

## Case Study: A First Look at the Data

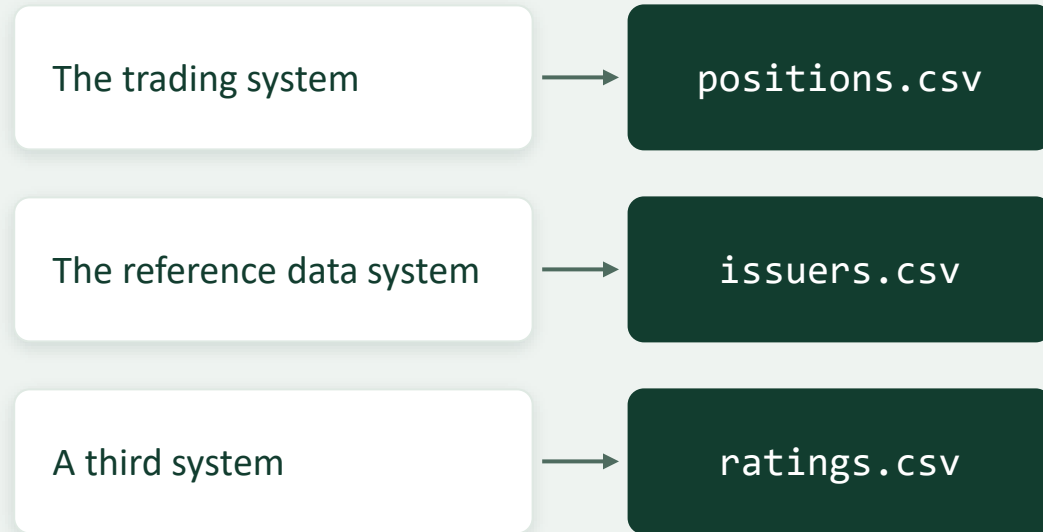
WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Three files, and nobody has checked they fit

Three systems, three files



**Built by different teams at different times. Nobody has checked that they fit together.**

Your job, before any report is built

## Know

each table: what one row is, how large it is, what each column holds

## Check

each table for missing values and repeated rows

## Test

that the key columns will connect the tables cleanly

## Report

what you found, and whether the tables are ready to join

*Nothing is joined today. The join is tomorrow's work.*

# Three tables and the keys that link them

## positions.csv

### What the desk holds

from the trading system

cusip

**ticker**

coupon

maturity

price

par\_held

ticker

## issuers.csv

### Who the issuers are

from the reference data system

**ticker**

issuer

sector

country

**rating**

rating

## The key

The column a connection is made on. Today both keys get tested.

## ratings.csv

### The rating scale

from a third system

**rating**

score

bucket

grade

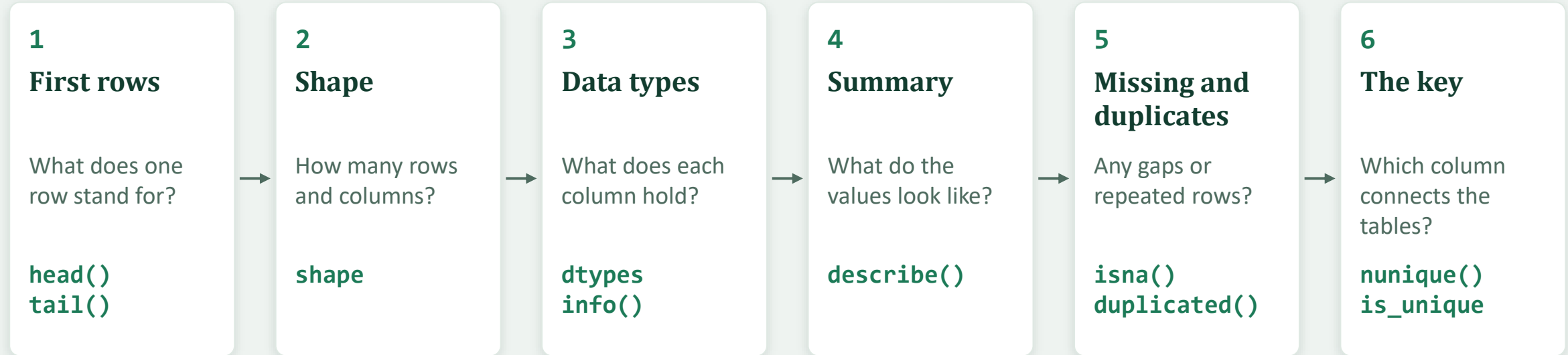
### positions to issuers, on ticker

Gives each bond its issuer's name, sector, and rating.

### issuers to ratings, on rating

Gives each rating its score, bucket, and grade.

# The same routine for every table



## Six Predict questions

Each shows code and asks what it will do.

Pause the video

Decide

Then run the cell

# Positions: one row is one bond held

**200**

rows, one bond held each

**6**

columns: 3 numbers, 3 text

**0**

missing values or duplicates

**489,000,000**

of face held in total

```
positions[['cusip', 'ticker',  
'maturity']].describe()
```

|        | cusip     | ticker | maturity   |
|--------|-----------|--------|------------|
| count  | 200       | 200    | 200        |
| unique | 200       | 115    | 196        |
| top    | 99080CAE8 | NEVX   | 2031-03-03 |
| freq   | 1         | 6      | 2          |

The CUSIP is distinct on every row. The ticker is not: 115 tickers cover the 200 bonds.

## The maturity arrived as text

```
pd.to_datetime(positions['maturity'])
```

All 200 convert: 2027-10-02 to 2056-08-23.

## The same checks in Excel

**=COUNTBLANK(E2:E201)** missing values in a column

**Remove Duplicates** counts repeats and deletes them

# Issuers: one row is one issuer

```
issuers['sector'].value_counts()
```

| sector         | issuers |
|----------------|---------|
| Materials      | 18      |
| Utilities      | 18      |
| Energy         | 17      |
| Industrials    | 17      |
| Telecom        | 16      |
| Transportation | 15      |
| Consumer       | 14      |
| Financials     | 14      |
| Health Care    | 12      |
| Technology     | 9       |

**150**

issuers, one row each

**5**

columns, all of them text

**0**

missing values or duplicate rows

**18**

distinct ratings, and 1 country

```
issuers['rating'].mean()
```

```
TypeError: Cannot perform reduction 'mean' with string dtype
```

## Why it failed

A rating is letters. BBB- and A+ cannot be added or averaged.

## Where the number is

The score of each rating is in ratings.csv. That is why the tables must be joined.

# The rating scale: one row is one rating

ratings.csv: 18 rows and 4 columns, small enough to read in full

| Investment Grade: 10 ratings, scores 1 to 10 |     |    |     |    |     |    |      |     |      | High Yield: 8 ratings, scores 11 to 18 |     |     |    |    |    |      |     |
|----------------------------------------------|-----|----|-----|----|-----|----|------|-----|------|----------------------------------------|-----|-----|----|----|----|------|-----|
| AAA                                          | AA+ | AA | AA- | A+ | A   | A- | BBB+ | BBB | BBB- | BB+                                    | BB  | BB- | B+ | B  | B- | CCC+ | CCC |
| 1                                            | 2   | 3  | 4   | 5  | 6   | 7  | 8    | 9   | 10   | 11                                     | 12  | 13  | 14 | 15 | 16 | 17   | 18  |
| AAA                                          | AA  |    | A   |    | BBB |    | BB   |     | B    |                                        | CCC |     |    |    |    |      |     |

Top to bottom: grade, rating, score, bucket

## 1 to 18

Every score once, in order,  
with no gap

## 7 and 2

Seven buckets and two  
grades, one grade per rating

## 0

Missing values or duplicate  
rows

## score

The only number in the table,  
and the one mean() needed

# Is each key unique?

positions: many rows

| cusip     | ticker | coupon |
|-----------|--------|--------|
| 99054CAF7 | NEVX   | 4.625  |
| 99054CAB6 | NEVX   | 6.250  |
| 99054CAC4 | NEVX   | 6.250  |
| 99054CAD2 | NEVX   | 6.125  |
| 99054CAG5 | NEVX   | 3.875  |
| 99054CAH3 | NEVX   | 5.250  |

issuers: one row

| ticker | issuer         | rating |
|--------|----------------|--------|
| NEVX   | Nevulex Health | BBB-   |

*Six NEVX bonds point to one issuer row.*

From a column that repeats to one that is unique

| Table     | Column | Unique? | Role                  |
|-----------|--------|---------|-----------------------|
| positions | cusip  | yes     | identifies one bond   |
| positions | ticker | no      | points to an issuer   |
| issuers   | ticker | yes     | identifies one issuer |
| issuers   | rating | no      | points to a rating    |
| ratings   | rating | yes     | identifies one rating |

```
print('ticker in issuers:', issuers['ticker'].is_unique)
print('rating in ratings:', ratings['rating'].is_unique)
```

```
ticker in issuers: True
rating in ratings: True
```

## The same test in Excel

**=COUNTIF(A:A, A2)**

A helper column, filtered  
for values above 1

**Remove Duplicates**

On a copy: if it removes  
nothing, the key is unique

# Every key has a match

Is every ticker of the positions in the issuers table?

The issuers table: 150 issuers

115

held by the desk. All 200 positions find their issuer here.

35

not held

```
found = positions['ticker'].isin(
    issuers['ticker'])
print(found.all())
True
```

Written down today, checked tomorrow

200

rows, after each join

13

columns at the end

489,000,000

of face, unchanged

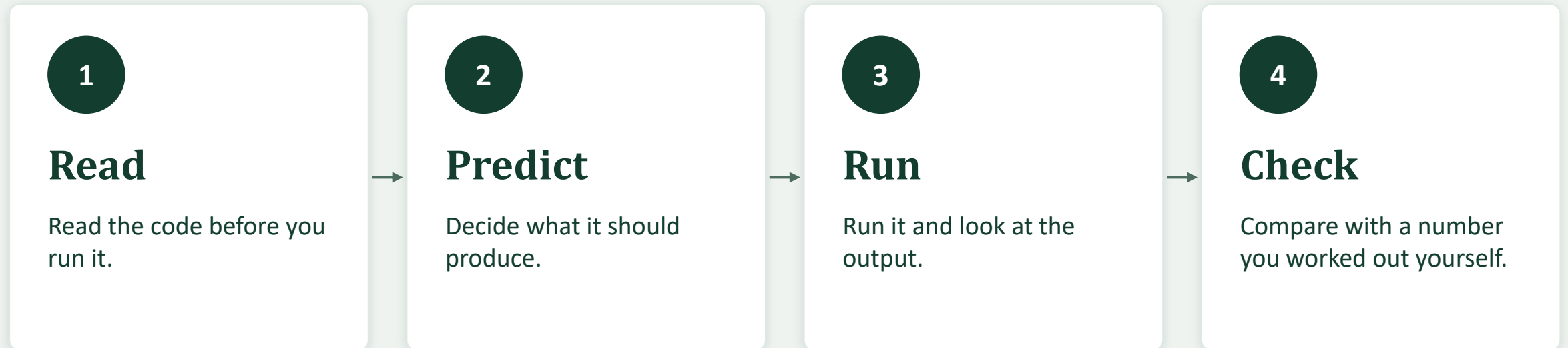
| Join                   | Key    | Expected result      |
|------------------------|--------|----------------------|
| positions to issuers   | ticker | 200 rows, 10 columns |
| that result to ratings | rating | 200 rows, 13 columns |

=NOT(ISNA(MATCH(B2, issuers!A:A, 0)))

In Excel: filled down, TRUE where the ticker is found.

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.



*Code that runs is not the same as code that is right.*

# What you can do now

- Give any new table the same first look: rows, shape, types, summary
- Count missing values and repeated rows before they distort a total
- Say which column identifies a row, and test that it is unique
- Test that every key has a match, with `isin()` and `all()`
- Write down what a join should return before you run it

**Tomorrow** Case study, part two: joining the three tables and answering the desk's questions